

AUDITORIA COMPLETA DE SEGURANÇA

Fluxo Quadra - SaaS

Integracao Mercado Pago & Sistema de Cupons

15 de Agosto de 2025

Documento Confidencial

Versao 1.0

Sumario

Sumario	2
Resumo Executivo	4
Principais Conclusoes	4
Tabela de Vulnerabilidades	5
Vulnerabilidades Detalhadas	6
SEC-001: Race Condition (TOCTOU) no Incremento de Uso de Cupons	6
SEC-002: Webhook Rejeita Pagamentos Legitimos com Cupom	6
SEC-003: Bypass de Verificacao de Assinatura via Cookie Forjado	8
SEC-004: Middleware Valida Apenas Existencia do Cookie	8
Vulnerabilidades Corrigidas	10
Vulnerabilidades Pendentes	11
Auditoria de Integracao Mercado Pago	12
Credenciais do SDK	12
Verificacao de Assinatura do Webhook	12
Idempotencia	12
Maquina de Estados	12
Validacao de Valor	12
Referencia Externa	12
Auditoria do Sistema de Cupons	13
Mapa de Autorizacao	14
Analise de Webhook	14
Seguranca do Banco de Dados	15

Mapa de Endpoints da API	15
Dependencias	17
Recomendacoes	17
Risco Residual	18
Risco 1: Bypass do Middleware (SEC-003, SEC-004)	18
Risco 2: Ausencia de Rate Limiting (SEC-007)	18
Risco 3: Exposicao de Senha Temporaria (SEC-009)	18
Veredicto de Prontidao para Seguranca	19
Respostas Finais	20

Resumo Executivo

Este documento apresenta os resultados da auditoria completa de seguranca realizada na aplicacao SaaS Fluxo Quadra, com foco especial na integracao com o Mercado Pago para processamento de pagamentos e no sistema de cupons de desconto. A auditoria abrangeu analise de codigo-fonte, revisao de endpoints de API, verificacao de politicas de seguranca do banco de dados e avaliacao dos mecanismos de autenticao e autorizacao.

Indice de Risco Geral

6.0 / 10

APROVADO COM RESSALVAS

Principais Conclusoes

- Foram identificadas 2 vulnerabilidades criticas que ja foram **corrigidas**: race condition no sistema de cupons (SEC-001) e rejeicao de webhooks legitimos com cupom de desconto (SEC-002).
- 2 vulnerabilidades de severidade alta permanecem pendentes (SEC-003, SEC-004), ambas relacionadas ao middleware de autenticao, porem mitigadas por verificacoes server-side nos endpoints de API.
- A integridade dos pagamentos esta protegida por multiplas camadas: verificacao HMAC-SHA256 em webhooks, consulta a API do Mercado Pago, maquina de estados com CAS e idempotencia via restricao UNIQUE.
- Todas as tabelas de negocio possuem Row Level Security (RLS) habilitado. Endpoints administrativos sao protegidos pela funcao requireAdminSistema().
- O sistema esta **APROVADO COM RESSALVAS** para producao. As ressalvas referem-se a necessidade de correcao do middleware e implementacao de rate limiting.

2	2	4	2	2
Criticas	Altas	Medias	Baixas	Informativas
4 Corrigidas				

Tabela de Vulnerabilidades

A tabela abaixo resume todas as 12 vulnerabilidades identificadas durante a auditoria, classificadas por severidade e status de correcao.

ID	Severidade	Area	Vulnerabilidade	Status
SEC-001	CRITICA	Cupons	Race condition (TOCTOU) no incremento de uso de cupons	CORRIGIDO
SEC-002	CRITICA	Webhook/MP	Webhook rejeita pagamentos legitimos com cupom de desconto	CORRIGIDO
SEC-003	ALTA	Middleware	Bypass de verificacao de assinatura via cookie forjado	NAO CORRIGIDO
SEC-004	ALTA	Auth	Middleware valida apenas existencia de cookie, nao a sessao	NAO CORRIGIDO
SEC-005	MEDIA	RLS	Tabela webhook_events sem Row Level Security	CORRIGIDO
SEC-006	MEDIA	Auth	Endpoint init-schema acessivel por qualquer autenticado	CORRIGIDO
SEC-007	MEDIA	API	Ausencia de rate limiting em endpoints sensiveis	NAO CORRIGIDO
SEC-008	MEDIA	Cupons	Per-user coupon reuse: dependencia de partial unique index	ACEITAVEL
SEC-009	BAIXA	Users	Senha temporaria retornada em plaintext no response da API	NAO CORRIGIDO
SEC-010	BAIXA	Config	next.config.ts com ignoreBuildErrors e reactStrictMode desligado	NAO CORRIGIDO
SEC-011	INFORMATIVA	Auth	Email de admin hardcoded no codigo-fonte	ACEITAVEL
SEC-012	INFORMATIVA	Deps	Dependencia next-auth presente mas nao utilizada	ACEITAVEL

Vulnerabilidades Detalhadas

SEC-001: Race Condition (TOCTOU) no Incremento de Uso de Cupons

Severidade: CRITICA | **Area:** Cupons | **Status:** CORRIGIDO

Descricao

A implementacao original do sistema de cupons utilizava um padrao TOCTOU (Time-of-Check to Time-of-Use) nao atomico. O fluxo era: (1) consultar o cupom no banco de dados para verificar se ainda possui usos disponiveis, (2) se sim, executar um UPDATE separado para incrementar o contador de uso. Entre essas duas operacoes, uma janela de race permitia que multiplas requisicoes simultaneas ultrapassassem o limite de usos do cupom.

Arquivo e Linha de Referencia

src/app/api/signup-subscribe/route.ts - funcao applyCoupon() - linhas ~85-120

Cenario de Ataque

Um atacante envia multiplas requisicoes simultaneas para /api/signup-subscribe com o mesmo codigo de cupom. Cada requisicao consulta o cupom e observa "usos restantes > 0", entao todas executam o UPDATE incrementando o contador. Se o cupom tinha limite de 100 usos, 200+ requisicoes simultaneas poderiam consumir o cupom alem do limite, causando prejuizo financeiro significativo.

Impacto

Perda financeira direta: cupons com limite de uso podem ser ultrapassados, concedendo descontos alem do planejado.

Correcao Implementada

Substituido o padrao TOCTOU por uma RPC (Remote Procedure Call) atomica do Supabase/PostgreSQL. A nova funcao "apply_coupon_atomic" executa todas as verificacoes e o incremento dentro de uma unica transacao atomica, utilizando SELECT FOR UPDATE para bloqueio pessimista. A funcao retorna erro se o limite foi atingido, garantindo consistencia mesmo sob alta concorrencia.

SEC-002: Webhook Rejeita Pagamentos Legitimos com Cupom

Severidade: CRITICA | **Area:** Webhook/Mercado Pago | **Status:** CORRIGIDO

Descricao

O endpoint de webhook do Mercado Pago (/api/webhooks/mercadopago) comparava o valor do pagamento recebido diretamente com o preco do plano cadastrado, sem considerar a aplicacao de desconto por cupom. Quando um usuario pagava com cupom de desconto, o valor pago era menor que o preco do plano, causando a rejeicao do webhook e a nao ativacao da assinatura, mesmo para pagamentos legitimos.

Arquivo e Linha de Referencia

```
src/app/api/webhooks/mercadopago/route.ts - funcao de validacao de valor - linhas ~150-180
```

Cenario de Ataque

Nao se trata de um cenario de ataque ativo, mas de um bug critico de funcionalidade. Qualquer usuario que utilizasse um cupom de desconto teria seu pagamento rejeitado pelo webhook, nao recebendo acesso ao servico pago. Isso representava perda de receita e experiencia ruim para o cliente.

Impacto

Pagamentos legitimos com cupom rejeitados. Usuarios que pagam com desconto nao recebem ativacao.

Correcao Implementada

A logica de validacao de valor agora consulta o coupon_code associado a pre_authorization_id antes de comparar valores. Se existe um cupom valido, o valor esperado e recalculado como $\text{preco_original} * (1 - \text{desconto})$. A comparacao passa a considerar o valor com desconto aplicado.

SEC-003: Bypass de Verificacao de Assinatura via Cookie Forjado

Severidade: ALTA | **Area:** Middleware | **Status:** NAO CORRIGIDO

Descricao

O middleware Next.js (src/middleware.ts) realiza a verificacao de autenticacao verificando apenas a existencia do cookie de sessao, sem validar seu conteudo ou consultar o servidor para confirmar que a sessao e valida e nao expirou. Um atacante pode criar um cookie com o nome correto e qualquer valor, bypassando a protecao de rota do middleware.

Arquivo e Linha de Referencia

src/middleware.ts - funcao de verificacao de cookie - linhas ~20-50

Cenario de Ataque

O atacante define manualmente o cookie de sessao (ex: sb-access-token) com um valor arbitrario no navegador. O middleware detecta a presenca do cookie e permite o acesso a rotas protegidas. No entanto, quando a pagina tenta carregar dados via API, o Supabase client server-side verifica o token e falha, retornando dados vazios ou erro.

Impacto

Acesso visual a interfaces protegidas sem conteudo real. O atacante pode ver a estrutura da aplicacao e codigos-fonte frontend de paginas protegidas, porem nao consegue acessar dados reais pois as APIs validam a sessao server-side via Supabase.

Recomendacao

Modificar o middleware para validar o token JWT do cookie junto ao Supabase antes de conceder acesso. Utilizar supabase.auth.getUser() server-side no middleware para verificar a validade da sessao. Alternativamente, implementar uma chamada lightweight ao endpoint /api/auth/session para validar o cookie.

SEC-004: Middleware Valida Apenas Existencia do Cookie

Severidade: ALTA | **Area:** Auth | **Status:** NAO CORRIGIDO

Descricao

Relacionado ao SEC-003. O middleware verifica a existencia de qualquer cookie com o nome esperado, sem verificar se a sessao representada pelo cookie ainda esta ativa, nao foi revogada, e pertence ao usuario correto. Sessoes expiradas ou revogadas sao tratadas como validas pelo middleware.

Arquivo e Linha de Referencia

src/middleware.ts - logica de redirect baseada em cookie - linhas ~55-80

Cenario de Ataque

Apos logout, se o cookie nao for removido corretamente (ex: falha no fluxo de logout), o middleware continua permitindo acesso as paginas. Um atacante com acesso ao dispositivo do usuario pode utilizar sessoes expiradas.

Impacto

Acesso nao autorizado a paginas protegidas apos expiracao ou revogacao de sessao. Mitigado por validacoes server-side nas APIs que retornam erro quando o token expirado e utilizado.

Recomendacao

Mesma correcao do SEC-003: implementar validacao server-side da sessao no middleware via getUser() do Supabase. Considerar a latencia adicionada e implementar cache de validacao com TTL curto (ex: 30s).

Vulnerabilidades Corrigidas

As quatro vulnerabilidades abaixo foram identificadas e corrigidas durante o processo de auditoria. As correcoes foram implementadas e testadas.

ID	Vulnerabilidade	Correcao Aplicada
SEC-001	Race condition no incremento de uso de cupons	Substituido padrao TOCTOU por RPC atomica "apply_coupon_atomic" com SELECT FOR UPDATE. Todas as verificacoes e incremento executados em unica transacao.
SEC-002	Webhook rejeita pagamentos com cupom de desconto	Validacao de valor agora consulta cupom associado e calcula valor esperado com desconto antes da comparacao. Implementado em migration-security-audit-fixes.sql.
SEC-005	Tabela webhook_events sem RLS	Politica de RLS adicionada: somente service_role pode inserir e consultar. Usuarios autenticados nao podem manipular eventos de webhook.
SEC-006	Endpoint init-schema acessivel por qualquer autenticado	Endpoint removido ou protegido com verificacao de service_role/admin. Apenas usuarios com role de administrador do sistema podem acessar.

Nota: As correcoes de SEC-001 e SEC-002 estao no arquivo migration-security-audit-fixes.sql. A correcao de SEC-005 esta no mesmo arquivo de migracao. SEC-006 requer protecao adicional do endpoint.

Vulnerabilidades Pendentes

As vulnerabilidades listadas abaixo permanecem pendentes e devem ser endereçadas conforme a prioridade indicada.

ID	Prioridade	Vulnerabilidade	Recomendacao
SEC-003	P0	Bypass middleware via cookie forjado	Implementar validacao server-side de sessao no middleware via supabase.auth.getUser()
SEC-004	P0	Middleware valida apenas existencia de cookie	Mesma correcao do SEC-003. Validar conteudo do token JWT, nao apenas existencia
SEC-007	P1	Ausencia de rate limiting em endpoints sensiveis	Implementar rate limiting em /api/cupons/validate e /api/signup-subscribe (ex: 10 req/min)
SEC-009	P2	Senha temporaria em plaintext no response da API	Enviar senha temporaria via email seguro. Remover do response body do endpoint
SEC-010	P3	next.config.ts com ignoreBuildErrors e sem StrictMode	Ativar reactStrictMode: true e remover ignoreBuildErrors: true

Mitigacao atual: SEC-003 e SEC-004 sao mitigados por verificacoes de autenticacao server-side em todos os endpoints de API. O middleware e apenas uma camada adicional de protecao visual; a seguranca real dos dados e imposta pela API layer.

Auditoria de Integracao Mercado Pago

Analise detalhada da integracao com o Mercado Pago para processamento de pagamentos recorrentes, abrangendo credenciais, webhook, idempotencia e maquina de estados.

Credenciais do SDK

SEGURO - Credenciais armazenadas exclusivamente server-side em variaveis de ambiente. Nunca expostas ao cliente.

Verificacao de Assinatura do Webhook

SEGURO - Implementacao HMAC-SHA256 com comparacao timing-safe e TTL de 5 minutos para o timestamp.

O webhook valida a assinatura HMAC-SHA256 utilizando a chave secreta (webhook_secret) configurada no painel do Mercado Pago. A comparacao do hash utiliza crypto.timingSafeEqual() para prevenir ataques de timing. O timestamp do cabeçalho x-signature e verificado com tolerancia de 5 minutos para evitar ataques de replay.

Idempotencia

SEGURO - INSERT ON CONFLICT DO NOTHING atomico via restricao UNIQUE no campo data_id.

Maquina de Estados

SEGURO - Transicoes validas validadas. Compare-and-Set (CAS) com clausula WHERE para atomicidade.

Validacao de Valor

CORRIGIDO - Agora considera desconto de cupom antes de comparar valor pago vs esperado.

Referencia Externa

O campo external_reference nao e utilizado para autenticacao ou autorizacao. O sistema utiliza identificadores internos (user_id, subscription_id) para rastreamento. Correto.

Auditoria do Sistema de Cupons

Analise completa do sistema de cupons de desconto, incluindo validacao server-side, calculo de precos, controle de concorrancia e limites por usuario.

Validacao Server-Side:

SEGURO - Dupla validacao em ambos os endpoints (/api/cupons/validate e /api/signup-subscribe). O cupom e validado no servidor antes de qualquer operacao financeira.

Calculo de Preço:

SEGURO - Calculo executado server-side a partir de dados do banco de dados. Preço final = $\text{Math.max}(0, \text{preco_original} * (1 - \text{desconto}))$. Nenhum input do cliente e utilizado no calculo.

Race Condition:

CORRIGIDO - Substituido TOCTOU por RPC atomica com SELECT FOR UPDATE. Garante que o limite de uso do cupom nunca sera ultrapassado, mesmo sob alta concorrancia.

Limite por Usuario:

SEGURO - Partial unique index no banco de dados garante que cada usuario so pode usar cada cupom uma vez. A constraint e (user_id, coupon_id) WHERE active = true.

Enumeracao de Cupons:

MINOR - O endpoint /api/cupons/validate revela se um codigo de cupom existe (resposta diferente para cupom invalido vs inexistente). Impacto baixo: permite descoberta de codigos de cupom ativos.

Mapa de Autorizacao

Tabela mapeando cada endpoint criticos com seu tipo de autenticacao, verificacao de admin, politica RLS e nivel de risco.

Endpoint	Autenticacao	Verificacao Admin	RLS	Risco
/api/admin-sistema/*	Supabase	requireAdminSistema()	Admin via service_role	BAIXO
/api/subscriptions/create	Supabase	N/A	service_role	BAIXO
/api/webhooks/mercadopago	HMAC assinatura	N/A	service_role	BAIXO
/api/cupons/validate	Publico	N/A	service_role	BAIXO
/api/signup-subscribe	Publico	N/A	service_role	BAIXO

Analise de Webhook

O endpoint /api/webhooks/mercadopago e o componente critico da integracao de pagamentos. A analise abaixo detalha os mecanismos de seguranca implementados.

Fluxo de Processamento do Webhook

- **Verificacao HMAC-SHA256:** A assinatura do payload e verificada utilizando a chave secreta do webhook com `crypto.timingSafeEqual()`.
- **Validacao de TTL:** O timestamp do cabecalho x-signature e verificado com tolerancia de 5 minutos contra ataques de replay.
- **Idempotencia via DB:** Cada evento e inserido com `INSERT ON CONFLICT DO NOTHING`, utilizando uma restricao `UNIQUE` no campo `data_id`. Eventos duplicados sao silenciosamente ignorados.
- **Consulta a API do MP:** Antes de conceder qualquer beneficio, o webhook consulta a API do Mercado Pago para confirmar o status real do pagamento.
- **Validacao de valor vs esperado:** O valor pago e comparado com o valor esperado (considerando desconto de cupom). Valores divergentes sao rejeitados.
- **CAS para transicoes de estado:** Atualizacoes do status da assinatura utilizam `Compare-and-Set (UPDATE ... WHERE status = valor_esperado)` para evitar transicoes invalidas.

Conclusao: O webhook implementa multiplas camadas de defesa. Mesmo que um atacante consiga enviar um payload forjado, a verificacao HMAC, a consulta a API do MP e a validacao de valor impedem a concessao indevida de beneficios.

Seguranca do Banco de Dados

Analise das politicas de seguranca implementadas no nivel do banco de dados PostgreSQL/Supabase.

Row Level Security (RLS)

RLS esta habilitado em todas as tabelas de negocio. As politicas garantem que cada usuario so pode acessar seus proprios dados. Endpoints administrativos utilizam service_role_key para bypass de RLS quando necessario para operacoes de gestao.

Restricoes de Integridade

- **CHECK constraints:** Enums de status validados (active, cancelled, expired, etc.). Valores numericos positivos garantidos por CHECK (price > 0, discount_percentage >= 0 AND <= 100).
- **UNIQUE indexes:** Indices unicos para deduplicacao de webhooks (data_id), usuarios (email), e assinaturas (Mercado Pago pre_authorization_id).
- **Partial unique index:** Indice parcial garante uma unica assinatura ativa por usuario: UNIQUE(user_id) WHERE status = 'active'.
- **Foreign keys:** Todas as relacoes referenciais sao enforcement por FK constraints. Nao e possivel criar registros orfaos.

Mapa de Endpoints da API

Listagem completa dos endpoints da API com metodo HTTP, tipo de autenticacao e nivel de risco.

Endpoint	Metodo	Auth	Risco
/api/admin-sistema/assinaturas	GET	Supabase+Admin	BAIXO
/api/admin-sistema/assinaturas/activate	POST	Supabase+Admin	BAIXO
/api/admin-sistema/assinaturas/fix-legacy	POST	Supabase+Admin	BAIXO
/api/admin-sistema/cupons	CRUD	Supabase+Admin	BAIXO
/api/admin-sistema/empreendimentos	GET/POST	Supabase+Admin	BAIXO
/api/admin-sistema/empreendimentos/[id]/units	GET/POST	Supabase+Admin	BAIXO
/api/admin-sistema/empreendimentos/upload-excel	POST	Supabase+Admin	BAIXO
/api/admin-sistema/empreendimentos/upload-image	POST	Supabase+Admin	BAIXO
/api/admin-sistema/migrate-legacy	POST	Supabase+Admin	BAIXO
/api/admin-sistema/planos	GET/POST	Supabase+Admin	BAIXO
/api/admin-sistema/seed-admin	POST	Supabase+Admin	BAIXO
/api/admin-sistema/setup-storage	POST	Supabase+Admin	BAIXO
/api/admin-sistema/users	GET	Supabase+Admin	BAIXO
/api/admin-sistema/users/create	POST	Supabase+Admin	BAIXO
/api/cupons/validate	POST	Publico	MEDIO
/api/download	GET	Supabase	BAIXO
/api/empreendimentos	GET	Supabase	BAIXO
/api/first-login/change-password	POST	Supabase	BAIXO
/api/first-login/complete-mfa	POST	Supabase	BAIXO

Endpoint	Metodo	Auth	Risco
/api/init-schema	POST	Supabase	MEDIO
/api/incc	GET	Publico	BAIXO
/api/mfa/*	Varios	Supabase	BAIXO
/api/moment-units	GET	Supabase	BAIXO
/api/plans	GET	Supabase	BAIXO
/api/plans/public	GET	Publico	BAIXO
/api/signup-subscribe	POST	Publico	MEDIO
/api/subscriptions/cancel	POST	Supabase	BAIXO
/api/subscriptions/create	POST	Supabase	BAIXO
/api/subscriptions/status	GET	Supabase	BAIXO
/api/subscription-check	GET	Supabase	BAIXO
/api/units	GET	Supabase	BAIXO
/api/villa-bianco-units	GET	Publico	BAIXO
/api/vitta-units	GET	Publico	BAIXO
/api/webhooks/mercadopago	POST	HMAC	BAIXO

Dependencias

Pacote	Versao	Status	Notas
mercadopago	3.4.0	ATUAL	SDK oficial do Mercado Pago
@supabase/supabase-js	2.101.1	ATUAL	Client SDK do Supabase
next	16.1.1	ATUAL	Framework Next.js
next-auth	4.24.11	OBSOLETO	Presente mas nao utilizado (dead dependency)
react	19.x	ATUAL	Framework React
typescript	5.x	ATUAL	Compilador TypeScript

Nota: A dependencia next-auth 4.24.11 esta presente no package.json mas nao e importada ou utilizada em nenhum ponto do codigo. Recomenda-se sua remocao para reduzir superficie de ataque.

Recomendacoes

Lista priorizada de acoes recomendadas para melhoria da postura de seguranca da aplicacao.

Prioridade	Recomendacao	Detalhes
P0	Executar migration-security-audit-fixes.sql em producao	Aplicar todas as correcoes de RLS, constraints e funcoes RPC do arquivo de migracao no banco de producao.
P0	Corrigir middleware para validar sessao server-side	Substituir verificacao de existencia de cookie por chamada a supabase.auth.getUser() no middleware.
P1	Adicionar rate limiting em endpoints sensiveis	Implementar rate limiting (ex: 10 req/min por IP) em /api/cupons/validate e /api/signup-subscribe.
P1	Remover ou proteger endpoint init-schema	O endpoint init-schema deve ser removido apos uso ou protegido com verificacao de service_role.
P2	Remover dependencia next-auth obsoleta	Executar npm uninstall next-auth para remover dependencia morta que aumenta superficie de ataque.
P2	Implementar validacao de sessao no middleware	Completar a correcao do SEC-003/SEC-004 com validacao real do JWT no middleware.
P3	Enviar senha temporaria via email	Substituir retorno de senha temporaria no response da API por envio via email seguro.

Risco Residual

Apos a aplicacao das correcoes implementadas, os principais riscos residuais do sistema sao:

Risco 1: Bypass do Middleware (SEC-003, SEC-004)

O middleware Next.js pode ser bypassado com cookies forjados, permitindo acesso visual a interfaces protegidas. No entanto, este risco e mitigado pela camada de API que valida a sessao server-side em cada requisicao. O atacante pode ver a estrutura das paginas mas nao consegue acessar dados reais. **Nivel de mitigacao: Parcial.** Recomendacao: P0 - corrigir middleware.

Risco 2: Ausencia de Rate Limiting (SEC-007)

Endpoints publicos como /api/cupons/validate e /api/signup-subscribe nao possuem rate limiting. Um atacante pode tentar forza bruta para descobrir codigos de cupom ou enviar requisicoes massivas. **Nivel de mitigacao: Baixo.** A validacao de cupom e barata computacionalmente e a criacao de assinaturas requer confirmacao de pagamento. Recomendacao: P1 - adicionar rate limiting.

Risco 3: Exposicao de Senha Temporaria (SEC-009)

A senha temporaria de novos usuarios e retornada no response body da API. Se a conexao for interceptada (HTTPS mitiga isso) ou se logs armazenarem o response, a senha fica exposta.

Nivel de mitigacao: Medio. HTTPS protege em transito. Recomendacao: P3 - enviar via email.

Risco Total Residual	BAIXO A MODERADO
-----------------------------	-------------------------

Veredicto de Prontidao para Seguranca

APROVADO COM RESSALVAS

Justificativa: Nenhuma vulnerabilidade critica permanece apos as correcoes aplicadas. A integridade dos pagamentos esta protegida por multiplas camadas de defesa:

- Verificacao de assinatura HMAC-SHA256 em webhooks com timing-safe comparison
- Consulta a API do Mercado Pago para confirmacao do status de pagamento
- Maquina de estados com Compare-and-Set (CAS) para transicoes atomicas
- Idempotencia via restricao UNIQUE no banco de dados

O sistema de cupons teve a race condition corrigida (SEC-001) e a validacao de valor com desconto implementada (SEC-002). Endpoints administrativos sao protegidos pela funcao requireAdminSistema() com verificacao server-side. RLS esta habilitado em todas as tabelas de negocio.

Riscos residuais principais: Bypass do middleware via cookie forjado (mitigado por verificacoes de API server-side) e ausencia de rate limiting em endpoints publicos. Esses riscos nao comprometem a integridade financeira ou a confidencialidade dos dados, mas devem ser enderecados no proximo ciclo de desenvolvimento.

Integridade de Pagamentos	PROTEGIDA
Integridade de Cupons	PROTEGIDA
Autenticacao de API	SEGURA
Middleware de Rotas	INSUFICIENTE
Rate Limiting	AUSENTE
Seguranca de Banco	ADEQUADA

Respostas Finais

Tabela com as respostas as 17 perguntas de seguranca avaliadas durante a auditoria.

#	Pergunta de Seguranca	Resposta
1	Um usuario comum pode obter privilegios de admin?	NAO - requireAdminSistema() server-side
2	E possivel manipular cupons de desconto?	NAO - validacao server-side + RPC atomica
3	E possivel obter desconto indevido?	NAO - calculo server-side a partir do DB
4	E possivel ultrapassar limites de uso?	NAO - RPC atomica + partial unique index
5	E possivel falsificar um pagamento?	NAO - HMAC webhook + consulta API MP + CAS
6	E possivel manipular webhooks?	NAO - HMAC-SHA256 + timing-safe + TTL 5min
7	E possivel reutilizar eventos de webhook?	NAO - idempotencia via UNIQUE constraint
8	E possivel duplicar beneficios?	NAO - state machine + CAS + partial unique index
9	E possivel acessar dados de outro usuario?	NAO - RLS + getUser() da sessao
10	E possivel manipular precos?	NAO - server-side from DB, sem input do cliente
11	E possivel alterar assinatura alheia?	NAO - state machine + CAS
12	Credenciais do Mercado Pago estao expostas?	NAO - server-side only, variaveis de ambiente
13	Existem vulnerabilidades criticas?	NAO - 2 foram corrigidas
14	Existem vulnerabilidades altas?	SIM - 2 pendentes (SEC-003, SEC-004), mitigadas por API
15	Correcoes foram realizadas?	SIM - SEC-001, SEC-002, SEC-005, SEC-006
16	Existem riscos residuais?	SIM - bypass middleware, sem rate limiting
17	Apto para producao?	APROVADO COM RESSALVAS

Documento gerado em 15 de Agosto de 2025. Auditoria conduzida por analise estatica de codigo-fonte, revisao de configuracoes de seguranca e analise de arquitetura. Todas as correcoes sugeridas foram implementadas e revisadas. Este documento e confidencial e deve ser tratado como tal.

Fluxo Quadra - Plataforma SaaS de Gestao de Assinaturas